threats that exploit vulnerabilities associated with the application's code. It's actually a cross-functional concern. If a Web app developer carefully checks out and removes the maximum possible vulnerabilities from the code during the development phase itself, then automatically, the security threat associated with the Web app reduces to a great extent. Some of the common targets for Web app attacks are content management systems, database administration tools and SaaS applications.

There are a couple of common measures like SSL encryption, using a secure host, etc, that can reduce the risk and vulnerability associated with Web applications. But at the same time, it's also necessary for Web apps to have their platforms updated, because hackers are always on the lookout for flaws that might be found in case of outdated versions of scripts. There should be a very strict and strong password policy for every Web application. Nowadays, hackers use sophisticated software (brute force attacks) to crack different passwords. So to protect against such attacks, passwords need to be complex. If by any chance the developer fails to secure the Web app on a particular e-retailer's site, the client's information can be stolen and ruin the relationship between the seller and buyer. In the worst case, it may even lead to legal proceedings; so caution is advocated.

Why Web applications are targeted by cyber criminals

Web apps are targeted mainly for the following reasons:

1. The inherent complexity of the source code for the Web apps increases the likelihood of unattended vulnerabilities, as well as the chances for malicious code manipulation.
2. It's easy to break the security of Web apps as most of the attacks can be automated by using Web bots that can be launched against tens of thousands of targets at the same time.
3. Hacking a Web app can yield high value rewards including sensitive data that can be collected using successful source code manipulation.

A few critical Web application security risks

Most hackers attack Web applications through cross-site scripting and SQL injection. Any flaws in the code, and a failure to sanitise application inputs and outputs, result in this. There are a bunch of other such critical security risks that cyber criminals exploit. As per the Open Web Application Security Project (OWASP), here are the ten most critical security risks of 2017.

1. SQL injection
2. Broken authentication
3. Sensitive data exposure
4. XML external entities (XXE)
5. Insecure direct object references

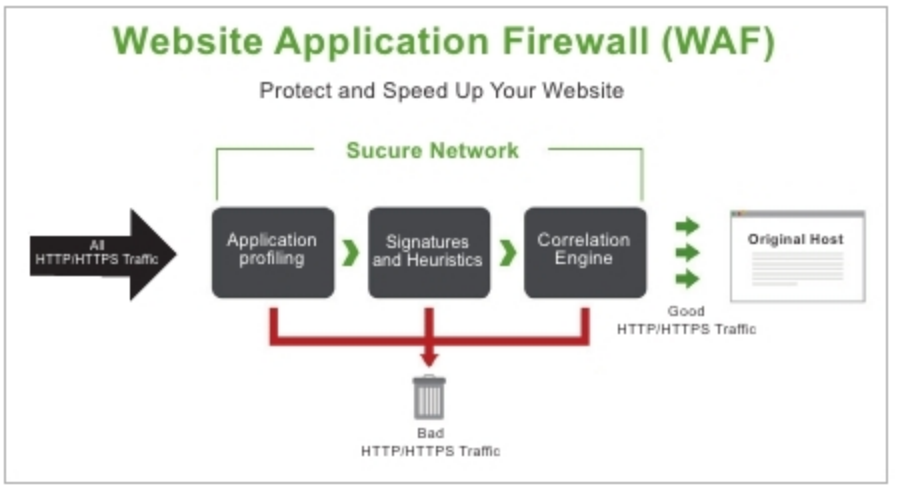

Figure 1: How a Web app firewall works (Image source: *googleimages.com*)

6. Security misconfiguration
7. Cross-site scripting (XSS)
8. Insecure de-serialisation
9. Using components with known vulnerabilities
10. Insufficient logging and monitoring

How to build a secure Web application

Here are some of the significant steps that lead to security-centric computer programming and help build a low-risk Web app.

Query parametrisation: There have been a couple of high-visibility attacks against different Web applications that have been traced back to attacks involving SQL injection, which has enabled the stealing of passwords. Governments, businesses and social network sites have all fallen victim to this, making it a universal issue. As a testament to the size of the problem, a recent 'Website Security Statistics Report' by White Hat found that 7 per cent of all the Web apps still contain SQL injections. While many of us consider it as a vendor issue, it's actually a developer and programming issue. Data fields, Web form comment boxes or any another area of a form that allows free data entry or open string input, can actually lead to this kind of vulnerability. This can also be transmitted through various non-visible Web elements like HTTP header values. This kind of simple insertion of malicious SQL code (which is impractical to validate and protect, at times) can be used to potentially wipe out entire databases or even run different malicious OS commands against the database. Hence, to stop SQL injection, developers must prevent all such untrusted inputs from being interpreted as SQL commands. This can be best implemented using query parametrisation.

Secure password storage: Indeed, the worst method for storing passwords is in the form of plain text; however, encryption is also not much better. Next on the list is MD5 or any such standard hashing algorithms that are, of course, fallible. Most of the hackers today have access to powerful computing resources that are not very expensive and allow them to create, or buy, rainbow tables. This helps different modestly strong passwords to be deciphered